

Explain Chocolate DP Paths

Yes. This is the **Cherry Pickup II / Maximum Chocolates** DP pattern, where **two robots move simultaneously** from the top row to the bottom row. The most important thing to understand is that your code is **not exploring one path**. At every row, it explores **9 combinations of moves** for the two robots, and memoization prevents recalculating the same `(row, robot1Column, robot2Column)` state.

1. First understand the picture

Suppose:

```
text

n = 4 rows
m = 4 columns

      col
      0  1  2  3
row 0 |  |  |  A  |  |  Robot 1 = A
      |  |  |  |  |
row 1 |  |  B  |  |  |  Robot 2 = B
      |  |  |  |  |
row 2 |  |  |  |  |
      |  |  |  |  |
row 3 |  |  |  |  |
      |  |  |  |  |
```

Actually, both robots **start on the same row**, but at different columns:

```
text

Robot 1 → column 0
Robot 2 → column m-1
```

So:

```
text

solve(0, 0, m-1)
```

means:

```
text

      Robot 1
      ↓
row 0 | R1 |  |  | R2 |
      |  |  |  |  |
row 1 |  |  |  |  |
      |  |  |  |  |
row 2 |  |  |  |  |
      |  |  |  |  |
row 3 |  |  |  |  |
      |  |  |  |  |
```

2. What does `solve(i, j1, j2)` mean?

This is the **most important concept**.

```
text

solve(i, j1, j2)
```

means:

```
text

Starting from row i,
Robot 1 is at column j1,
Robot 2 is at column j2,

what is the maximum number of chocolates
they can collect from row i → last row?
```

So:

```
text

solve(2, 1, 3)
```

means:

```
text

Current row = 2

Robot 1 → column 1
Robot 2 → column 3

Find maximum chocolates from here
until the bottom.
```

3. Why only 3 moves for each robot?

Each robot can move:

```
text

      ↓
    ↙ ↓ ↘

j-1  j  j+1
```

Therefore:

```
text

Robot 1:
j1 → j1-1
j1 → j1
j1 → j1+1

Robot 2:
j2 → j2-1
j2 → j2
j2 → j2+1
```

Each robot has:

```
text

3 choices
```

Therefore:

```
text

3 × 3 = 9 combinations
```

4. The 9 combinations

Suppose:

```
text

j1 = 1
j2 = 2
```

Robot 1 has:

```
text

1 + (-1) = 0
1 + 0 = 1
1 + 1 = 2
```

Robot 2 has:

```
text

2 + (-1) = 1
2 + 0 = 2
2 + 1 = 3
```

Therefore:

text

	Robot 2		
	1	2	3
Robot 1 = 0	(0,1)	(0,2)	(0,3)
Robot 1 = 1	(1,1)	(1,2)	(1,3)
Robot 1 = 2	(2,1)	(2,2)	(2,3)

These are the 9 recursive calls.

5. How exactly your nested loops generate them

Your code:

C++

```
for (int dj1 = -1; dj1 <= 1; dj1++) {
    for (int dj2 = -1; dj2 <= 1; dj2++) {
        int ans = curr + solve(i + 1, j1 + dj1, j2 + dj2, ...);
        maxi = max(maxi, ans);
    }
}
```

Let's execute it manually.

Suppose:

text

```
i  = 0
j1 = 1
j2 = 2
```

Outer loop starts

text

```
dj1 = -1
```

Robot 1 moves:

text

```
1 + (-1) = 0
```

Now inner loop runs completely.

Inner loop 1

text

```
dj2 = -1
```

Robot 2:

text

```
2 + (-1) = 1
```

Therefore:

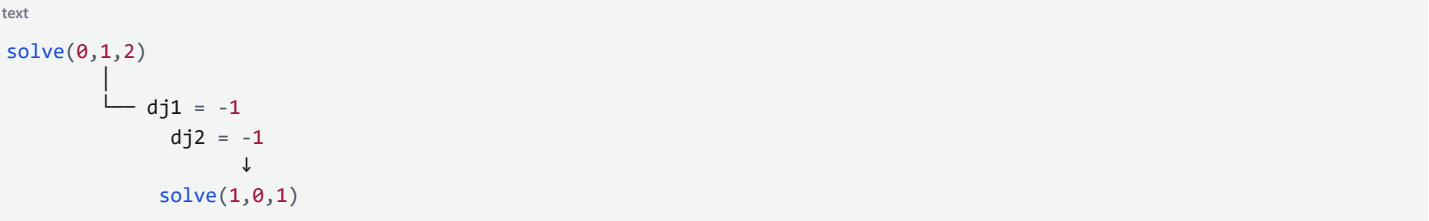


Diagram:

text

Row 0

R1 = 1 R2 = 2

 ↙ ↙

 R1 R2

 ↓ ↓

Row 1

R1 = 0 R2 = 1

Inner loop 2

text

dj2 = 0

Robot 1 remains:

text

0

Robot 2:

text

2 + 0 = 2

Call:

text

solve(1,0,2)

Diagram:

text

Row 0

R1 = 1 R2 = 2

 ↙ ↓

 ↓ ↓

Row 1

R1 = 0 R2 = 2

Inner loop 3

text

dj2 = 1

Robot 2:

text

2 + 1 = 3

Call:

text

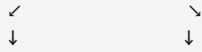
solve(1,0,3)

Diagram:

text

Row 0

R1 = 1 R2 = 2



Row 1

R1 = 0 R2 = 3

6. Then outer loop changes

Now:

text

dj1 = 0

Robot 1:

text

1 + 0 = 1

Again, inner loop starts from:

text

dj2 = -1

So:

text

`solve(1,1,1)`

Then:

text

`solve(1,1,2)`

Then:

text

`solve(1,1,3)`

7. Finally dj1 = 1

Robot 1:

text

1 + 1 = 2

Again:

text

dj2 = -1 → `solve(1,2,1)`

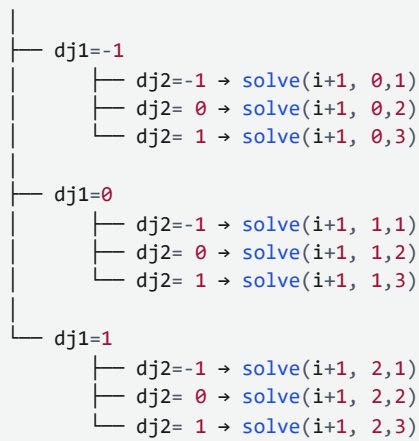
dj2 = 0 → `solve(1,2,2)`

dj2 = 1 → `solve(1,2,3)`

Therefore the complete sequence is:

text

```
solve(i, 1, 2)
```



That's the 9-way branching.

8. Now let's see the complete recursion tree

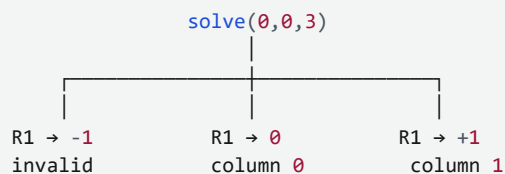
Suppose:

text

```
solve(0, 0, 3)
```

Then:

text



Because Robot 1 starts at column 0.

For Robot 1:

text

```
j1 = 0
```

```
j1 - 1 = -1 ❌
j1    = 0   ✅
j1 + 1 = 1   ✅
```

Robot 2 starts at 3:

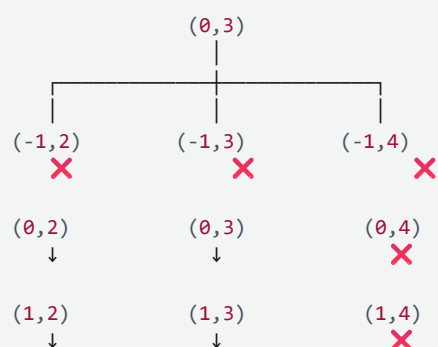
text

```
j2 = 3
```

```
j2 - 1 = 2   ✅
j2    = 3   ✅
j2 + 1 = 4   ❌
```

So only valid combinations are:

text



Therefore from the starting state:

```
text
solve(0,0,3)
```

only 4 valid combinations exist.

9. Why are invalid paths returned as `-1e9`?

Your code:

```
C++
if (j1 < 0 || j1 >= m || j2 < 0 || j2 >= m)
    return -1e9;
```

Imagine:

```
text
m = 4

valid columns:

0 1 2 3
```

If:

```
text
j1 = -1
```

that's outside the grid.

So:

```
text
solve(...)
  ↓
j1 = -1
  ↓
invalid
  ↓
return -1e9
```

Why `-1e9`?

Because we're doing:

```
C++
max(...)
```

and we don't want an invalid path to accidentally become the maximum.

10. Now the chocolate calculation

This part is also critical:

```
C++
int curr = (j1 == j2)
    ? grid[i][j1]
    : grid[i][j1] + grid[i][j2];
```

Suppose:

```
text
grid:

      0  1  2  3
0  [ 2  1  3  4 ]
1  [ 5  6  2  1 ]
2  [ 1  2  8  3 ]
3  [ 4  2  1  7 ]
```

Starting:

text

```
R1 = column 0
R2 = column 3
```

So:

text

```
grid[0][0] = 2
grid[0][3] = 4
```

Therefore:

text

```
curr = 2 + 4
      = 6
```

11. What if both robots land on the same cell?

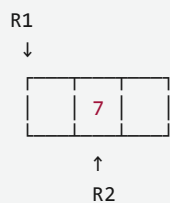
Suppose:

text

```
j1 = 1
j2 = 1
```

Then:

text



They cannot collect the same chocolate twice.

Therefore:

C++

```
if (j1 == j2)
    curr = grid[i][j1];
```

NOT:

C++

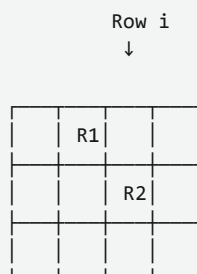
```
grid[i][j1] + grid[i][j2]
```

Otherwise we'd count the same cell twice.

12. The most important recursion idea

Suppose we are here:

text



At this point:

text

```
current chocolates
+
maximum chocolates obtainable from next row
```

That's exactly what this line does:

```
C++
int ans = curr + solve(i + 1, j1 + dj1, j2 + dj2, ...);
```

Conceptually:

```
text
ANSWER OF CURRENT STATE
=
CURRENT CHOCOLATES
+
BEST ANSWER FROM NEXT STATE
```

13. Let's follow ONE complete path

Take:

```
text
R1 starts = 0
R2 starts = 3
```

Suppose we choose:

```
text
Row 0:
R1 → +1
R2 → -1

Row 1:
R1 → +1
R2 → -1

Row 2:
R1 → 0
R2 → -1
```

Then:

```
text
Row 0:

R1 = 0 → 1
R2 = 3 → 2

    ↓

Row 1:

R1 = 1 → 2
R2 = 2 → 1

    ↓

Row 2:

R1 = 2 → 2
R2 = 1 → 0

    ↓

Row 3

STOP
```

So one path is:

```
text
(R1, R2)

(0, 3)
↓
(1, 2)
↓
(2, 1)
↓
(2, 0)
```

Each arrow represents one pair:

```
text
(dj1, dj2)
```

For example:

```
text
(0, 3) → (1, 2)

dj1 = +1
dj2 = -1
```

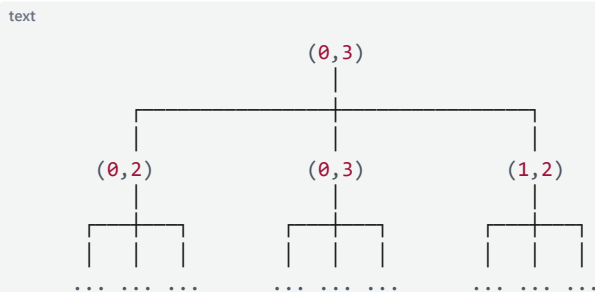
14. But recursion doesn't follow only one path

This is where beginners commonly misunderstand the code.

It does NOT do:

```
text
Path 1
↓
Path 2
↓
Path 3
```

Instead it does:



It explores all possible valid combinations.

Then:

```
C++
maxi = max(maxi, ans);
```

selects the best one.

15. What exactly does `maxi` do?

Suppose from one state we get:

```
text
Move 1 → 20 chocolates
Move 2 → 27 chocolates
Move 3 → 19 chocolates
Move 4 → 31 chocolates
Move 5 → 25 chocolates
...
```

Then:

```
text
maxi
```

```
initial:
-1e9

after move 1:
20

after move 2:
27

after move 3:
27

after move 4:
31

after move 5:
31

...

```

Finally:

```
text
maxi = 31

```

So:

```
C++
maxi = max(maxi, ans);

```

means:

Among all 9 possible next movements, keep the one that gives the maximum chocolates.

16. The base case

When:

```
C++
i == n - 1

```

we are on the **last row**.

Example:

```
text
Row 0
↓
Row 1
↓
Row 2
↓
Row 3 ← LAST ROW

```

There is nowhere else to move.

Therefore recursion stops.

```
C++
if (i == n - 1) {
    if (j1 == j2)
        return grid[i][j1];
    else
        return grid[i][j1] + grid[i][j2];
}

```

Example:

text

Last row:

4	2	8	7
---	---	---	---

R1	R2
↓	↓
2	8

Answer:

text

$$2 + 8 = 10$$

If:

text

R1 → column 2

R2 → column 2

then:

text

8

not:

text

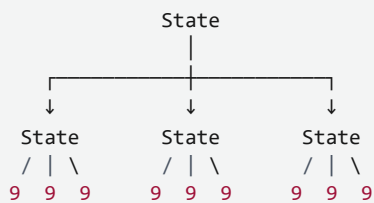
$$8 + 8 = 16$$

17. Now the REAL role of memoization

Without memoization, the recursion becomes enormous.

Think about this:

text



Many branches eventually reach the **same state**.

For example:

text

`solve(2, 1, 2)`

might be reached through multiple different paths.

But the answer from:

text

`solve(2,1,2)`

is always the same.

So:

C++

```
if (dp[i][j1][j2] != -1)
    return dp[i][j1][j2];
```

means:

text

Have we already solved this state?

YES



Return stored answer

NO



Calculate recursively

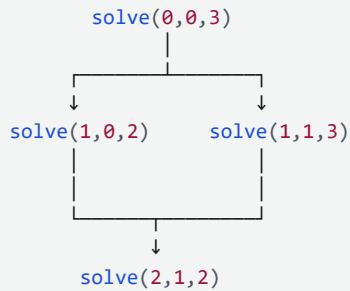


Store answer

18. Visualizing memoization

Imagine:

text



Later another path also reaches:

text

`solve(2,1,2)`

Without memoization:

text

`solve(2,1,2)`



calculate everything again

With memoization:

text

`solve(2,1,2)`



`dp[2][1][2]` already calculated



return `dp[2][1][2]`

That is the entire purpose of:

C++

`dp[i][j1][j2]`

19. What does the DP array represent?

Your DP:

C++

```
vector<vector<vector<int>>> dp(
    n,
    vector<vector<int>>(m, vector<int>(m, -1))
);
```

creates:

text

`dp[n][m][m]`

Think of it as:

text

```
row
  |
  v
dp[i][j1][j2]

i = current row
j1 = Robot 1 column
j2 = Robot 2 column
```

For example:

text

```
dp[2][1][3]
```

means:

text

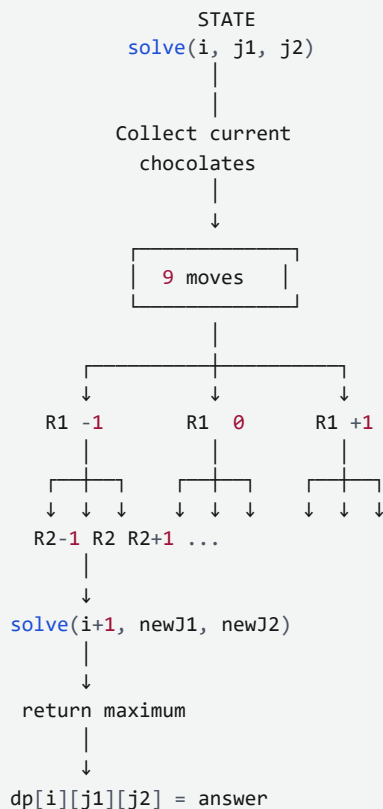
Maximum chocolates obtainable
starting from:

```
row = 2
Robot 1 = column 1
Robot 2 = column 3
```

20. Complete state-transition diagram

This is the cleanest way to remember the algorithm:

text



21. The complete mathematical DP idea

Your recursive state is:

text

```
dp[i][j1][j2]
```

```
i → current row
j1 → Robot 1 position
j2 → Robot 2 position
```

Transition:

```

text
dp[i][j1][j2]
=
current chocolates
+
max(
    dp[i+1][j1-1][j2-1],
    dp[i+1][j1-1][j2],
    dp[i+1][j1-1][j2+1],

    dp[i+1][j1][j2-1],
    dp[i+1][j1][j2],
    dp[i+1][j1][j2+1],

    dp[i+1][j1+1][j2-1],
    dp[i+1][j1+1][j2],
    dp[i+1][j1+1][j2+1]
)

```

The code's nested loops are simply a compact way of generating these 9 transitions.

22. One very important point: the recursion tree is NOT the DP table

This distinction matters.

Recursion tree

Represents:

```

text
all possible paths/moves

```

It can contain the same state multiple times.

DP table

Represents:

```

text
unique states

```

For example:

```

text
Recursion:

Path A → solve(2,1,2)
Path B → solve(2,1,2)
Path C → solve(2,1,2)
Path D → solve(2,1,2)

```

But DP stores only:

```

text
dp[2][1][2]

```

once.

That's why memoization converts the exponential-looking recursion into a manageable DP.

23. Complexity

There are:

```

text
n × m × m

```

possible states.

For every state, we try:

```

text
3 × 3 = 9

```

transitions.

Therefore:

text

Time:

$O(n \times m^2 \times 9)$

$= O(n \times m^2)$

Space:

text

DP:

$O(n \times m^2)$

Recursion stack:

$O(n)$

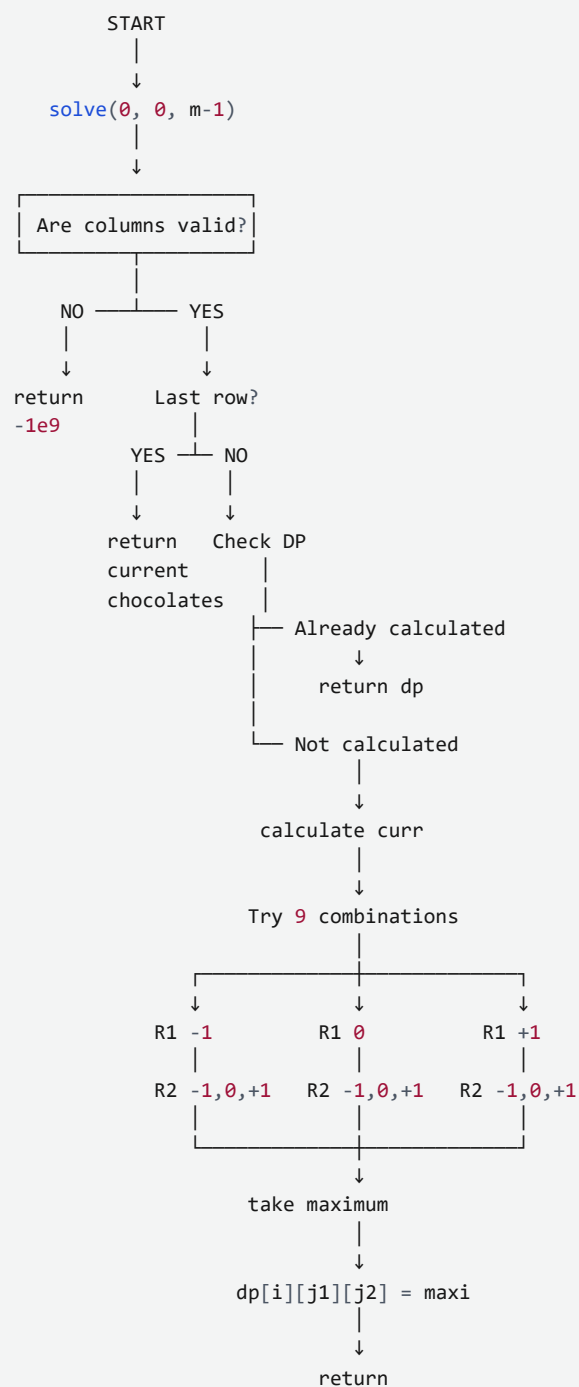
Total:

text

$O(n \times m^2)$ space

24. The whole algorithm in one diagram

text



25. The mental model you should use

Don't think:

"The robot is moving around and recursion is finding a path."

Think this instead:

text

I have TWO robots.

At every row I know:

current row
Robot 1 column
Robot 2 column

From this state there are at most:

3 choices for Robot 1
×
3 choices for Robot 2
=
9 choices

For every choice:

current chocolates
+
best possible chocolates from next row

Take the maximum.

Store that answer.

So the core is simply:

text

STATE
↓
9 POSSIBLE NEXT STATES
↓
RECURSIVELY FIND BEST ANSWER
↓
TAKE MAXIMUM
↓
MEMORIZE

And the key state is:

text

solve(i, j1, j2)

i = row
j1 = Robot 1 column
j2 = Robot 2 column

One subtle correction to your intuition: there aren't really "two independent paths." The state is a **pair of synchronized paths**—both robots move from row `i` to row `i+1` together. That's why one transition requires **both** `dj1` **and** `dj2`, producing 9 combinations.